Summer Internship Training

(June 2026 - July 2026)

on

Mastering Data Structures using Java

Concepts to Real-world Applications

Submitted by

(Name of Student1) - Tejesh Kadiyam **Registration Number: 12407122**

Under the Guidance of

Dr. Om Prakash Yadav
Associate Professor

School of Computer Science and Engineering



As Example

Project Title : Smart Library management System

Project Overview

The Smart Library Management System is a standalone console-based application developed using Core Java. It is designed to automate the day-to-day operations of a library, such as managing books, registering students, issuing and returning books, maintaining waiting lists, calculating overdue fines, and generating reports. The project follows a clean layered architecture and uses Java Collections, File Handling, and Object-Oriented Programming principles to provide an efficient and organized library management solution

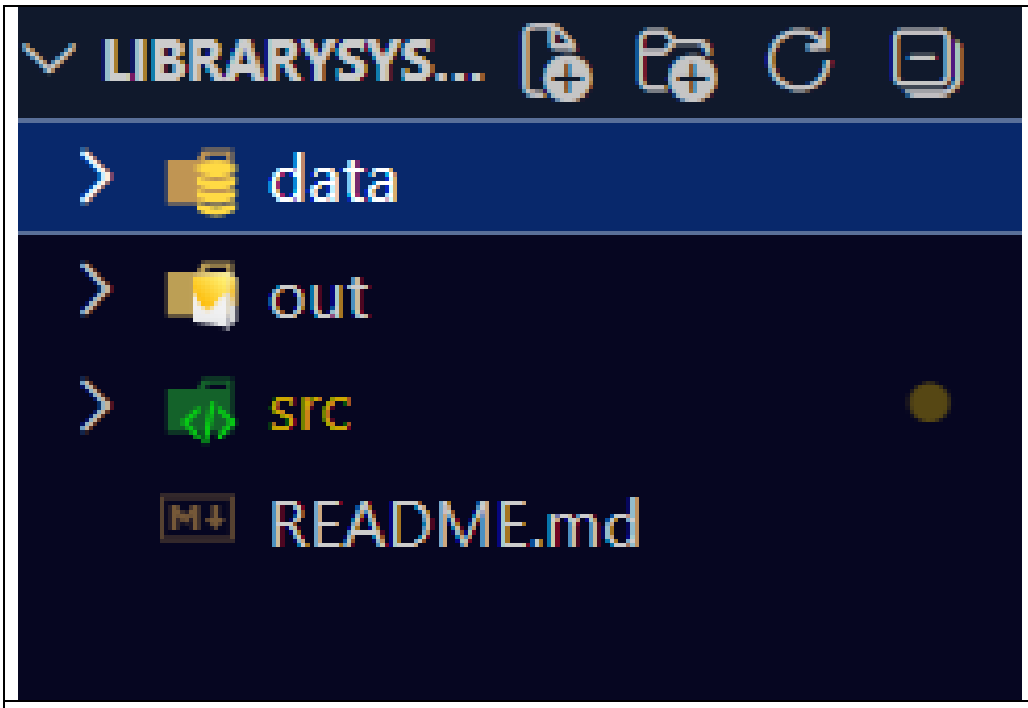
Project Requirements

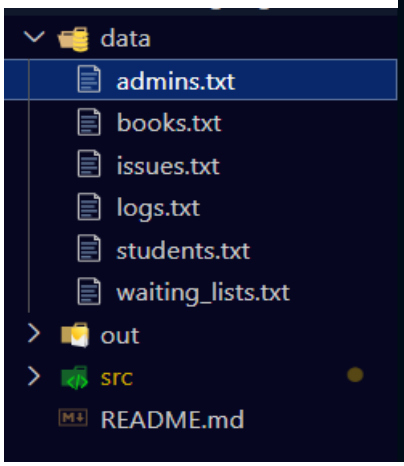
1. **Programming Language**: JAVA
2. **Libraries Used**:
    - <java.util> - ArrayList, HashMap, Queue, Stack, Scanner, List, Map
    - <ava.io> - File handling (File, BufferedReader, BufferedWriter, FileReader, FileWriter)
    - <Java.time> - Date and time handling, due dates, fine calculation
    - <java.util.regex> - Input validation using regular expressions (email, phone number)

      <java.util.concurrent> - Time calculations using TimeUnit
3. **Dataset (if any)** :
    - This project does not use any external dataset. Instead, it uses self-created sample data stored in pipe-separated text files. These files act as the application's persistent storage and are loaded into memory when the application starts.

Implementation Steps

Code:


LIBRARYSYS...
data
out
src
README.md




```java
// Source code is decompiled from a .class file using FernFlower decompiler (from Intellij IDEA).
package config;

public final class Constants {
   public static final String DATA_DIR = "src/data";
   public static final String BOOKS_FILE = "src/data/books.txt";
   public static final String STUDENTS_FILE = "src/data/students.txt";
   public static final String ADMINS_FILE = "src/data/admins.txt";
   public static final String ISSUES_FILE = "src/data/issues.txt";
   public static final String WAITING_LISTS_FILE = "src/data/waiting_lists.txt";
   public static final String LOGS_FILE = "src/data/logs.txt";
   public static final int BORROW_DURATION_DAYS = 14;
   public static final double FINE_PER_DAY = (double)5.0F;
   public static final int MAX_BORROW_LIMIT = 3;
   public static final String BOOK_ID_PREFIX = "BK";
   public static final String STUDENT_ID_PREFIX = "ST";
   public static final String ADMIN_ID_PREFIX = "AD";
   public static final String ISSUE_ID_PREFIX = "IS";
   public static final String DATE_FORMAT = "yyyy-MM-dd";
   public static final String DELIMITER = "\\|";
   public static final String LIST_DELIMITER = ",";

   private Constants() {
   }
}
```

```java
// Source code is decompiled from a .class file using FernFlower decompiler (from Intellij ID
package exception;

public class BookNotFoundException extends Exception {
   public BookNotFoundException(String var1) {
      super(var1);
   }
}
```

```java
// Source code is decompiled from a .class file using FernFlower decompiler (from Intellij IDEA).
package exception;

public class BookUnavailableException extends Exception {
   public BookUnavailableException(String var1) {
      super(var1);
   }
}
```

```java
// Source code is decompiled from a .class file using FernFlower decompiler (from Intellij IDEA).
package exception;

public class BorrowLimitExceededException extends Exception {
   public BorrowLimitExceededException(String var1) {
      super(var1);
   }
}
```

```java
// Source code is decompiled from a .class file using FernFlower decompiler (from Intellij IDEA).
package exception;

public class StudentNotFoundException extends Exception {
   public StudentNotFoundException(String var1) {
      super(var1);
   }
}
```

```java
// Source code is decompiled from a .class file using FernFlower decompiler (from Intellij IDEA).
package model;

public class Admin {
   private String adminId;
   private String username;
   private String password;

   public Admin() {
   }

   public Admin(String var1, String var2, String var3) {
      this.adminId = var1;
      this.username = var2;
      this.password = var3;
   }

   public String getAdminId() {
      return this.adminId;
   }

   public void setAdminId(String var1) {
      this.adminId = var1;
   }

   public String getUsername() {
      return this.username;
   }

   public void setUsername(String var1) {
      this.username = var1;
   }

   public String getPassword() {
      return this.password;
   }
```

```java
37
38      public void setPassword(String var1) {
39         this.password = var1;
40      }
41   }
42
```

```java
// Source code is decompiled from a .class file using FernFlower decompiler (from Intellij IDEA).
package model;

public class Book {
   private String bookId;
   private String isbn;
   private String title;
   private String author;
   private BookCategory category;
   private String publisher;
   private int publicationYear;
   private int totalCopies;
   private int availableCopies;
   private String shelfNumber;

   public Book() {
   }

   public Book(String var1, String var2, String var3, String var4, BookCategory var5, String var6, int var7, int var8, int var9, Strin
      this.bookId = var1;
      this.isbn = var2;
      this.title = var3;
      this.author = var4;
      this.category = var5;
      this.publisher = var6;
      this.publicationYear = var7;
      this.totalCopies = var8;
      this.availableCopies = var9;
      this.shelfNumber = var10;
   }

   public boolean isAvailable() {
      return this.availableCopies > 0;
   }

   public void issueCopy() {
```

```java
   public void issueCopy() {
      if (this.availableCopies <= 0) {
         throw new IllegalStateException("No copies available to issue.");
      } else {
         --this.availableCopies;
      }
   }

   public void returnCopy() {
      if (this.availableCopies >= this.totalCopies) {
         throw new IllegalStateException("All copies are already available.");
      } else {
         ++this.availableCopies;
      }
   }

   public String toString() {
      return "Book{bookId='" + this.bookId + '\'' + ", isbn='" + this.isbn + '\'' + ", title='" + this.title + '\'' + ", author='" + th
   }

   public String getBookId() {
      return this.bookId;
   }

   public void setBookId(String var1) {
      this.bookId = var1;
   }

   public String getIsbn() {
      return this.isbn;
   }

   public void setIsbn(String var1) {
      this.isbn = var1;
   }
```

```
70      }
71
72      public String getTitle() {
73          return this.title;
74      }
75
76      public void setTitle(String var1) {
77          this.title = var1;
78      }
79
80      public String getAuthor() {
81          return this.author;
82      }
83
84      public void setAuthor(String var1) {
85          this.author = var1;
86      }
87
88      public BookCategory getCategory() {
89          return this.category;
90      }
91
92      public void setCategory(BookCategory var1) {
93          this.category = var1;
94      }
95
96      public String getPublisher() {
97          return this.publisher;
98      }
99
100     public void setPublisher(String var1) {
101         this.publisher = var1;
102     }
103
```

```
103
104     public int getPublicationYear() {
105         return this.publicationYear;
106     }
107
108     public void setPublicationYear(int var1) {
109         this.publicationYear = var1;
110     }
111
112     public int getTotalCopies() {
113         return this.totalCopies;
114     }
115
116     public void setTotalCopies(int var1) {
117         this.totalCopies = var1;
118     }
119
120     public int getAvailableCopies() {
121         return this.availableCopies;
122     }
123
124     public void setAvailableCopies(int var1) {
125         this.availableCopies = var1;
126     }
127
128     public String getShelfNumber() {
129         return this.shelfNumber;
130     }
131
132     public void setShelfNumber(String var1) {
133         this.shelfNumber = var1;
134     }
135
136     public Book copy() {
137         return new Book(this.bookId, this.isbn, this.title, this.author, this.category, this.publisher, this.publicationYear, this.totalC
```

```
Click to add a breakpoint
138     }
139 }
140
```

```
// Source code is decompiled from a .class file using FernFlower decompiler (from Intellij IDEA).
package model;

public enum BookCategory {
    PROGRAMMING,
    DATABASE,
    NETWORKING,
    OPERATING_SYSTEM,
    MATHEMATICS,
    SCIENCE,
    HISTORY,
    FICTION;

    private BookCategory() {
    }

    public static BookCategory fromString(String var0) {
        if (var0 != null && !var0.trim().isEmpty()) {
            return valueOf(var0.trim().toUpperCase());
        } else {
            throw new IllegalArgumentException("Category cannot be empty.");
        }
    }
}
```

```java
// Source code is decompiled from a .class file using FernFlower decompiler (from Intellij IDEA).
package model;

public enum EntityType {
    BOOK,
    STUDENT,
    ISSUE_RECORD;

    private EntityType() {
    }
}
```

```java
// Source code is decompiled from a .class file using FernFlower decompiler (from Intellij IDEA).
package model;

public class IssueRecord {
    private String issueId;
    private String studentId;
    private String bookId;
    private String issueDate;
    private String dueDate;
    private String returnDate;
    private double fineAmount;

    public IssueRecord() {
    }

    public IssueRecord(String var1, String var2, String var3, String var4, String var5, String var6, double var7) {
Click to add a breakpoint sueId = var1;
        this.studentId = var2;
        this.bookId = var3;
        this.issueDate = var4;
        this.dueDate = var5;
        this.returnDate = var6;
        this.fineAmount = var7;
    }

    public boolean isActive() {
        return this.returnDate == null || this.returnDate.trim().isEmpty();
    }

    public boolean isOverdue(String var1) {
        return this.isActive() && var1.compareTo(this.dueDate) > 0;
    }

    public String toString() {
        return "IssueRecord{issueId='" + this.issueId + '\'' + ", studentId='" + this.studentId + '\'' + ", bookId='" + this.bookId + '\
```

```java
    }

    public String getIssueId() {
        return this.issueId;
    }

    public void setIssueId(String var1) {
        this.issueId = var1;
    }

    public String getStudentId() {
        return this.studentId;
    }

    public void setStudentId(String var1) {
        this.studentId = var1;
    }

    public String getBookId() {
        return this.bookId;
    }

    public void setBookId(String var1) {
        this.bookId = var1;
    }

    public String getIssueDate() {
        return this.issueDate;
    }

    public void setIssueDate(String var1) {
        this.issueDate = var1;
    }

    public String getDueDate() {
```

```java
    }

    public String getDueDate() {
        return this.dueDate;
    }

    public void setDueDate(String var1) {
        this.dueDate = var1;
    }

    public String getReturnDate() {
        return this.returnDate;
    }

    public void setReturnDate(String var1) {
        this.returnDate = var1;
    }

    public double getFineAmount() {
        return this.fineAmount;
    }

    public void setFineAmount(double var1) {
        this.fineAmount = var1;
    }

    public IssueRecord copy() {
        return new IssueRecord(this.issueId, this.studentId, this.bookId, this.issueDate, this.dueDate, this.returnDate, this.fineAmount);
    }
}
```

```java
// Source code is decompiled from a .class file using FernFlower decompiler (from Intellij IDEA).
package model;

public enum OperationType {
   ADD,
   UPDATE,
   DELETE,
   ISSUE,
   RETURN,
   REGISTER_STUDENT;

   private OperationType() {
   }
}

```

```java
// Source code is decompiled from a .class file using FernFlower decompiler (from Intellij IDEA).
package model;

import java.util.ArrayList;
import java.util.List;

public class Student {
   private String studentId;
   private String name;
   private String department;
   private String phoneNumber;
   private String emailAddress;
   private final List<String> borrowedBookIds;

   public Student() {
      this.borrowedBookIds = new ArrayList();
   }

   public Student(String var1, String var2, String var3, String var4, String var5, List<String> var6) {
      this.studentId = var1;
      this.name = var2;
      this.department = var3;
      this.phoneNumber = var4;
      this.emailAddress = var5;
      this.borrowedBookIds = new ArrayList(var6);
   }

   public void addBorrowedBook(String var1) {
      if (!this.borrowedBookIds.contains(var1)) {
         this.borrowedBookIds.add(var1);
      }

   }

   public void removeBorrowedBook(String var1) {
      this.borrowedBookIds.remove(var1);
   }
}
```

```
38
39      public int getBorrowedCount() {
40          return this.borrowedBookIds.size();
41      }
42
43      public String toString() {
44          return "Student{studentId='" + this.studentId + '\'' + ", name='" + this.name + '\'' + ", department='" + this.department + '
45      }
46
47      public String getStudentId() {
48          return this.studentId;
49      }
50
51      public void setStudentId(String var1) {
52          this.studentId = var1;
53      }
54
55      public String getName() {
56          return this.name;
57      }
58
59      public void setName(String var1) {
60          this.name = var1;
61      }
62
63      public String getDepartment() {
64          return this.department;
65      }
66
67      public void setDepartment(String var1) {
68          this.department = var1;
69      }
70
71      public String getPhoneNumber() {
72          return this.phoneNumber;
```

```
71      public String getPhoneNumber() {
72          return this.phoneNumber;
73      }
74
75      public void setPhoneNumber(String var1) {
76          this.phoneNumber = var1;
77      }
78
79      public String getEmailAddress() {
80          return this.emailAddress;
81      }
82
83      public void setEmailAddress(String var1) {
84          this.emailAddress = var1;
85      }
86
87      public List<String> getBorrowedBookIds() {
88          return new ArrayList(this.borrowedBookIds);
89      }
90
91      public Student copy() {
92          return new Student(this.studentId, this.name, this.department, this.phoneNumber, this.emailAddress, this.borrowedBookIds);
93      }
94  }
95
```

```java
// Source code is decompiled from a .class file using FernFlower decompiler (from Intellij IDEA).
package model;

public class UndoOperation {
   private final OperationType operationType;
   private final EntityType entityType;
   private final String previousState;
   private final String currentState;
   private final String description;

   public UndoOperation(OperationType var1, EntityType var2, String var3, String var4, String var5) {
      this.operationType = var1;
      this.entityType = var2;
      this.previousState = var3;
      this.currentState = var4;
      this.description = var5;
   }

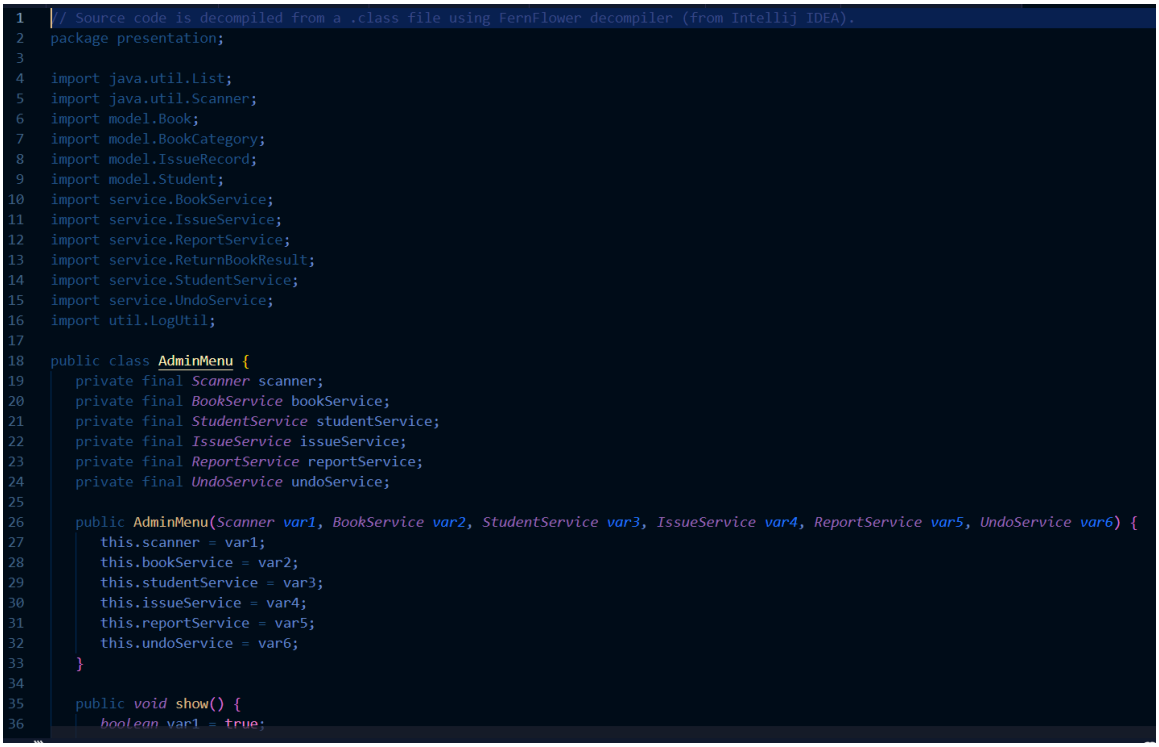
   public OperationType getOperationType() {
      return this.operationType;
   }

   public EntityType getEntityType() {
      return this.entityType;
   }

   public String getPreviousState() {
      return this.previousState;
   }

   public String getCurrentState() {
      return this.currentState;
   }

   public String getDescription() {
      return this.description;
```



```java
// Source code is decompiled from a .class file using FernFlower decompiler (from Intellij IDEA).
package presentation;

import java.util.List;
import java.util.Scanner;
import model.Book;
import model.BookCategory;
import model.IssueRecord;
import model.Student;
import service.BookService;
import service.IssueService;
import service.ReportService;
import service.ReturnBookResult;
import service.StudentService;
import service.UndoService;
import util.LogUtil;

public class AdminMenu {
   private final Scanner scanner;
   private final BookService bookService;
   private final StudentService studentService;
   private final IssueService issueService;
   private final ReportService reportService;
   private final UndoService undoService;

   public AdminMenu(Scanner var1, BookService var2, StudentService var3, IssueService var4, ReportService var5, UndoService var6) {
      this.scanner = var1;
      this.bookService = var2;
      this.studentService = var3;
      this.issueService = var4;
      this.reportService = var5;
      this.undoService = var6;
   }

   public void show() {
      boolean var1 = true;
```

```
38      while(var1) {
39          this.printMenu();
40          int var2 = this.readIntInput("Enter your choice: ");
41
42          try {
43              switch (var2) {
44                  case 1:
45                      this.addBook();
46                      break;
47                  case 2:
48                      this.updateBook();
49                      break;
50                  case 3:
51                      this.deleteBook();
52                      break;
53                  case 4:
54                      this.searchBooks();
55                      break;
56                  case 5:
57                      this.viewAllBooks();
58                      break;
59                  case 6:
60                      this.registerStudent();
61                      break;
62                  case 7:
63                      this.searchStudents();
64                      break;
65                  case 8:
66                      this.viewAllStudents();
67                      break;
68                  case 9:
69                      this.issueBook();
70                      break;
71                  case 10:
```



```
                    break;
                case 10:
                    this.returnBook();
                    break;
                case 11:
                    this.reportService.generateFullReport();
                    break;
                case 12:
                    this.undoLastAction();
                    break;
                case 13:
                    LogUtil.info("Admin logged out.");
                    System.out.println("Logged out successfully.");
                    var1 = false;
                    break;
                default:
                    System.out.println("Invalid choice. Please try again.");
            }
        } catch (Exception var4) {
            System.out.println("Error: " + var4.getMessage());
        }
    }

}

private void printMenu() {
    System.out.println("\n--- Administrator Menu ---");
    System.out.println(" 1. Add Book");
    System.out.println(" 2. Update Book");
    System.out.println(" 3. Delete Book");
    System.out.println(" 4. Search Book");
```

```
101         System.out.println(" 5. View All Books");
102         System.out.println(" 6. Register Student");
103         System.out.println(" 7. Search Student");
104         System.out.println(" 8. View All Students");
105         System.out.println(" 9. Issue Book");
106         System.out.println("10. Return Book");
107         System.out.println("11. Generate Reports");
108         System.out.println("12. Undo Last Action");
109         System.out.println("13. Logout");
110     }
111
112     private void addBook() {
113         System.out.println("\n--- Add Book ---");
114         String var1 = this.readStringInput("ISBN: ");
115         String var2 = this.readStringInput("Title: ");
116         String var3 = this.readStringInput("Author: ");
117         BookCategory var4 = this.readCategory();
118         String var5 = this.readStringInput("Publisher: ");
119         int var6 = this.readIntInput("Publication Year: ");
120         int var7 = this.readIntInput("Total Copies: ");
121         String var8 = this.readStringInput("Shelf Number: ");
122         Book var9 = this.bookService.addBook(var1, var2, var3, var4, var5, var6, var7, var8);
123         System.out.println("Book added successfully!");
124         ConsoleFormatter.printBook(var9);
125     }
126
127     private void updateBook() throws Exception {
128         System.out.println("\n--- Update Book ---");
129         String var1 = this.readStringInput("Book ID: ");
130         Book var2 = this.bookService.getBookById(var1);
131         System.out.println("Leave blank to keep current value.");
132         String var3 = this.readOptionalString("ISBN", var2.getIsbn());
133         String var4 = this.readOptionalString("Title", var2.getTitle());
134         String var5 = this.readOptionalString("Author", var2.getAuthor());
```



```
        BookCategory var6 = this.readOptionalCategory(var2.getCategory());
        String var7 = this.readOptionalString("Publisher", var2.getPublisher());
        int var8 = this.readOptionalInt("Publication Year", var2.getPublicationYear());
        int var9 = this.readOptionalInt("Total Copies", var2.getTotalCopies());
        String var10 = this.readOptionalString("Shelf Number", var2.getShelfNumber());
        Book var11 = this.bookService.updateBook(var1, var3, var4, var5, var6, var7, var8, var9, var10);
        System.out.println("Book updated successfully!");
        ConsoleFormatter.printBook(var11);
    }

    private void deleteBook() throws Exception {
        System.out.println("\n--- Delete Book ---");
        String var1 = this.readStringInput("Book ID: ");
        this.bookService.deleteBook(var1);
        System.out.println("Book deleted successfully.");
    }

    private void searchBooks() {
        System.out.println("\n--- Search Books ---");
        String var1 = this.readStringInput("Enter search keyword (or press Enter for all): ");
        List var2 = this.bookService.searchBooks(var1);
        this.displayBookList(var2);
    }

    private void viewAllBooks() {
        System.out.println("\n--- All Books ---");
        this.displayBookList(this.bookService.getAllBooks());
    }

    private void registerStudent() {
        System.out.println("\n--- Register Student ---");
        String var1 = this.readStringInput("Name: ");
        String var2 = this.readStringInput("Department: ");
```

```
        String var3 = this.readStringInput("Phone (10 digits): ");
        String var4 = this.readStringInput("Email: ");
        Student var5 = this.studentService.registerStudent(var1, var2, var3, var4);
        System.out.println("Student registered successfully!");
        ConsoleFormatter.printStudent(var5);
    }

    private void searchStudents() {
        System.out.println("\n--- Search Students ---");
        String var1 = this.readStringInput("Enter search keyword (or press Enter for all): ");
        List var2 = this.studentService.searchStudents(var1);
        this.displayStudentList(var2);
    }

    private void viewAllStudents() {
        System.out.println("\n--- All Students ---");
        this.displayStudentList(this.studentService.getAllStudents());
    }

    private void issueBook() throws Exception {
        System.out.println("\n--- Issue Book ---");
        String var1 = this.readStringInput("Student ID: ");
        String var2 = this.readStringInput("Book ID: ");
        IssueRecord var3 = this.issueService.issueBook(var1, var2);
        System.out.println("Book issued successfully!");
        ConsoleFormatter.printIssueRecord(var3);
    }

    private void returnBook() throws Exception {
        System.out.println("\n--- Return Book ---");
        String var1 = this.readStringInput("Student ID: ");
        String var2 = this.readStringInput("Book ID: ");
        ReturnBookResult var3 = this.issueService.returnBook(var1, var2);
        IssueRecord var4 = var3.getIssueRecord();
```

```
    }

    private String resolveStudentName(String var1) {
        if (var1 == null) {
            return null;
        } else {
            Student var2 = this.database.findStudentById(var1);
            return var2 != null ? var2.getName() : var1;
        }
    }

    private void recordUndoForIssue(String var1, String var2, Book var3, Student var4, IssueRecord var5, String var6, String var7) {
        String var8 = buildIssueUndoState(var1, var2, "");
        String var9 = buildIssueUndoState(BookRepository.serialize(var3), StudentRepository.serialize(var4), IssueRepository.serialize(v
        this.undoService.pushOperation(new UndoOperation(OperationType.ISSUE, EntityType.ISSUE_RECORD, var8, var9, "Issued book " + var6
    }

    private void recordUndoForReturn(String var1, String var2, String var3, Book var4, Student var5, IssueRecord var6, String var7, Str
        String var9 = buildIssueUndoState(var1, var2, var3);
        String var10 = buildIssueUndoState(BookRepository.serialize(var4), StudentRepository.serialize(var5), IssueRepository.serialize(
        this.undoService.pushOperation(new UndoOperation(OperationType.RETURN, EntityType.ISSUE_RECORD, var9, var10, "Returned book " + 
    }

    private IssueRecord findActiveIssueRecord(String var1, String var2) {
        for(IssueRecord var4 : this.database.getIssueRecords()) {
            if (var4.getStudentId().equals(var1) && var4.getBookId().equals(var2) && var4.isActive()) {
                return var4;
            }
        }

        return null;
    }
```

```
    private Book findBookOrThrow(String var1) throws BookNotFoundException {
        Book var2 = this.database.findBookById(var1);
        if (var2 == null) {
            throw new BookNotFoundException("Book not found with ID: " + var1);
        } else {
            return var2;
        }
    }

    private void persistAll() {
        this.bookRepository.save();
        this.studentRepository.save();
        this.issueRepository.save();
    }

    public static class BorrowedBookView {
        private final IssueRecord record;
        private final String bookTitle;

        public BorrowedBookView(IssueRecord var1, String var2) {
            this.record = var1;
            this.bookTitle = var2;
        }

        public IssueRecord getRecord() {
            return this.record;
        }

        public String getBookTitle() {
            return this.bookTitle;
        }
    }
}
```

```java
// Source code is decompiled from a .class file using FernFlower decompiler (from Intellij IDEA).
package service;

import model.IssueRecord;

public class ReturnBookResult {
   private final IssueRecord issueRecord;
   private final String waitingStudentId;
   private final String waitingStudentName;
   private final String bookTitle;
   private final String bookId;

   public ReturnBookResult(IssueRecord var1, String var2, String var3, String var4, String var5) {
      this.issueRecord = var1;
      this.waitingStudentId = var2;
      this.waitingStudentName = var3;
      this.bookTitle = var4;
      this.bookId = var5;
   }

   public IssueRecord getIssueRecord() {
      return this.issueRecord;
   }

   public boolean hasWaitingListNotification() {
      return this.waitingStudentId != null;
   }

   public String getWaitingStudentId() {
      return this.waitingStudentId;
   }

   public String getWaitingStudentName() {
      return this.waitingStudentName;
   }
```

```java
   public String getBookTitle() {
      return this.bookTitle;
   }

   public String getBookId() {
      return this.bookId;
   }
}
```

```java
// Source code is decompiled from a .class file using FernFlower decompiler (from Intellij IDEA).
package service;

import exception.StudentNotFoundException;
import java.util.ArrayList;
import java.util.Comparator;
import java.util.List;
import model.EntityType;
import model.OperationType;
import model.Student;
import model.UndoOperation;
import repository.LibraryDatabase;
import repository.StudentRepository;
import util.IdGenerator;
import util.LogUtil;
import util.ValidationUtil;

public class StudentService {
   private final LibraryDatabase database;
   private final StudentRepository studentRepository;
   private final UndoService undoService;

   public StudentService(LibraryDatabase var1, StudentRepository var2, UndoService var3) {
      this.database = var1;
      this.studentRepository = var2;
      this.undoService = var3;
   }

   public Student registerStudent(String var1, String var2, String var3, String var4) {
      this.validateStudentInput(var1, var2, var3, var4);
      String var5 = IdGenerator.generateStudentId(this.database);
      Student var6 = new Student(var5, var1.trim(), var2.trim(), var3.trim(), var4.trim(), new ArrayList());
      this.database.addStudent(var6);
      this.studentRepository.save();
      this.undoService.pushOperation(new UndoOperation(OperationType.REGISTER_STUDENT, EntityType.STUDENT, "", StudentRepository.seria
      LogUtil.info("Student registered: " + var5 + " - " + var1);
```

```java
   }

   private Student findStudentOrThrow(String var1) throws StudentNotFoundException {
      Student var2 = this.database.findStudentById(var1);
      if (var2 == null) {
         throw new StudentNotFoundException("Student not found with ID: " + var1);
      } else {
         return var2;
      }
   }

   private void validateStudentInput(String var1, String var2, String var3, String var4) {
      ValidationUtil.validateNotEmpty(var1, "Name");
      ValidationUtil.validateNotEmpty(var2, "Department");
      ValidationUtil.validateNotEmpty(var3, "Phone number");
      ValidationUtil.validateNotEmpty(var4, "Email address");
      if (!ValidationUtil.isValidPhone(var3)) {
         throw new IllegalArgumentException("Phone number must be exactly 10 digits.");
      } else if (!ValidationUtil.isValidEmail(var4)) {
         throw new IllegalArgumentException("Invalid email address format.");
      }
   }
}
```

```java
// Source code is decompiled from a .class file using FernFlower decompiler (from Intellij IDEA).
package util;

import java.time.LocalDate;
import java.time.format.DateTimeFormatter;
import java.time.temporal.ChronoUnit;

public final class DateUtil {
   private static final DateTimeFormatter FORMATTER = DateTimeFormatter.ofPattern("yyyy-MM-dd");

   private DateUtil() {
   }

   public static String getCurrentDate() {
      return LocalDate.now().format(FORMATTER);
   }

   public static String calculateDueDate(String var0) {
      LocalDate var1 = LocalDate.parse(var0, FORMATTER);
      return var1.plusDays(14L).format(FORMATTER);
   }

   public static long daysBetween(String var0, String var1) {
      LocalDate var2 = LocalDate.parse(var0, FORMATTER);
      LocalDate var3 = LocalDate.parse(var1, FORMATTER);
      return ChronoUnit.DAYS.between(var2, var3);
   }

   public static double calculateFine(String var0, String var1) {
      if (var1 == null || var1.trim().isEmpty()) {
         var1 = getCurrentDate();
      }

      if (var1.compareTo(var0) <= 0) {
         return (double)0.0F;
      } else {
```

```
28
29      public static double calculateFine(String var0, String var1) {
30          if (var1 == null || var1.trim().isEmpty()) {
31              var1 = getCurrentDate();
32          }
33
34          if (var1.compareTo(var0) <= 0) {
35              return (double)0.0F;
36          } else {
37              long var2 = daysBetween(var0, var1);
38              return (double)var2 * (double)5.0F;
39          }
40      }
41
42      public static String formatDate(LocalDate var0) {
43          return var0.format(FORMATTER);
44      }
45
46      public static boolean isValidDate(String var0) {
47          try {
48              LocalDate.parse(var0, FORMATTER);
49              return true;
50          } catch (Exception var2) {
51              return false;
52          }
53      }
54  }
55
```



```java
// Source code is decompiled from a .class file using FernFlower decompiler (from Intellij IDEA).
package util;

import java.io.File;
import java.nio.charset.Charset;
import java.nio.charset.StandardCharsets;

public final class FileUtil {
   public static final Charset FILE_CHARSET;

   private FileUtil() {
   }

   public static void ensureParentDirectory(File var0) {
      File var1 = var0.getParentFile();
      if (var1 != null && !var1.exists()) {
         var1.mkdirs();
      }

   }

   static {
      FILE_CHARSET = StandardCharsets.UTF_8;
   }
}

```

```java
// Source code is decompiled from a .class file using FernFlower decompiler (from Intellij IDEA).
package util;

import java.util.List;
import java.util.function.Function;
import model.Book;
import model.IssueRecord;
import model.Student;
import repository.LibraryDatabase;

public final class IdGenerator {
   private IdGenerator() {
   }

   public static String generateBookId(LibraryDatabase var0) {
      return generateId("BK", var0.getBooks(), Book::getBookId);
   }

   public static String generateStudentId(LibraryDatabase var0) {
      return generateId("ST", var0.getStudents(), Student::getStudentId);
   }

   public static String generateIssueId(LibraryDatabase var0) {
      return generateId("IS", var0.getIssueRecords(), IssueRecord::getIssueId);
   }

   private static <T> String generateId(String var0, List<T> var1, Function<T, String> var2) {
      int var3 = 0;

      for(Object var5 : var1) {
         String var6 = (String)var2.apply(var5);
         if (var6 != null && var6.startsWith(var0)) {
            try {
               int var7 = Integer.parseInt(var6.substring(var0.length()));
               var3 = Math.max(var3, var7);
            } catch (NumberFormatException var8) {
```

```java
   }

   public static String generateIssueId(LibraryDatabase var0) {
      return generateId("IS", var0.getIssueRecords(), IssueRecord::getIssueId);
   }

   private static <T> String generateId(String var0, List<T> var1, Function<T, String> var2) {
      int var3 = 0;

      for(Object var5 : var1) {
         String var6 = (String)var2.apply(var5);
         if (var6 != null && var6.startsWith(var0)) {
            try {
               int var7 = Integer.parseInt(var6.substring(var0.length()));
               var3 = Math.max(var3, var7);
            } catch (NumberFormatException var8) {
            }
         }
      }

      return String.format("%s%03d", var0, var3 + 1);
   }
}
```

```java
// Source code is decompiled from a .class file using FernFlower decompiler (from Intellij IDEA).
package util;

import java.io.BufferedWriter;
import java.io.File;
import java.io.IOException;
import java.nio.file.Files;
import java.nio.file.StandardOpenOption;
import java.time.LocalDateTime;
import java.time.format.DateTimeFormatter;

public final class LogUtil {
   private static final DateTimeFormatter TIMESTAMP_FORMATTER = DateTimeFormatter.ofPattern("yyyy-MM-dd HH:mm:ss");

   private LogUtil() {
   }

   public static void info(String var0) {
      log("INFO", var0, false);
   }

   public static void warning(String var0) {
      log("WARNING", var0, true);
   }

   public static void error(String var0) {
      log("ERROR", var0, true);
   }

   private static void log(String var0, String var1, boolean var2) {
      String var3 = LocalDateTime.now().format(TIMESTAMP_FORMATTER);
      String var4 = var3 + "|" + var0 + "|" + var1;
      if (var2) {
         System.out.println("[" + var0 + "] " + var1);
      }

```

```java
   private static void appendToFile(String var0) {
      try {
         File var1 = new File("src/data/logs.txt");
         FileUtil.ensureParentDirectory(var1);
         BufferedWriter var2 = Files.newBufferedWriter(var1.toPath(), FileUtil.FILE_CHARSET, StandardOpenOption.CREATE, StandardOpenOp
         Throwable var3 = null;

         try {
            var2.write(var0);
            var2.newLine();
         } catch (Throwable var13) {
            var3 = var13;
            throw var13;
         } finally {
            if (var2 != null) {
               if (var3 != null) {
                  try {
                     var2.close();
                  } catch (Throwable var12) {
                     var3.addSuppressed(var12);
                  }
               } else {
                  var2.close();
               }
            }

         }
      } catch (IOException var15) {
         System.err.println("Failed to write log: " + var15.getMessage());
      }

   }
}
```

```java
// Source code is decompiled from a .class file using FernFlower decompiler (from Intellij IDEA).
package util;

public final class ValidationUtil {
   private ValidationUtil() {
   }

   public static boolean isNotEmpty(String var0) {
      return var0 != null && !var0.trim().isEmpty();
   }

   public static void validateNotEmpty(String var0, String var1) {
      if (!isNotEmpty(var0)) {
         throw new IllegalArgumentException(var1 + " cannot be empty.");
      }
   }

   public static boolean isValidEmail(String var0) {
      return !isNotEmpty(var0) ? false : var0.matches("^[A-Za-z0-9+_.-]+@[A-Za-z0-9.-]+\\.[A-Za-z]{2,}$");
   }

   public static boolean isValidPhone(String var0) {
      return !isNotEmpty(var0) ? false : var0.matches("^[0-9]{10}$");
   }

   public static boolean isValidId(String var0, String var1) {
      return !isNotEmpty(var0) ? false : var0.matches("^" + var1 + "\\d{3}$");
   }
}

```

```java
// Source code is decompiled from a .class file using FernFlower decompiler
import presentation.LibraryApplication;

public class Main {
   public Main() {
   }

   public static void main(String[] var0) {
      LibraryApplication var1 = new LibraryApplication();
      var1.start();
   }
}

```

OUTPUT: :

```
PS C:\Users\TEJESH\Desktop\LibrarySystem> java -cp out Main
==============================================
     SMART LIBRARY MANAGEMENT SYSTEM
==============================================

--- Main Menu ---
1. Administrator Login
2. Student Login
3. Exit
Enter your choice: 1

--- Administrator Login ---
Username: admin
Password: admin123
Welcome, admin!

--- Administrator Menu ---
 1. Add Book
 2. Update Book
 3. Delete Book
 4. Search Book
 5. View All Books
 6. Register Student
 7. Search Student
 8. View All Students
 9. Issue Book
10. Return Book
11. Generate Reports
12. Undo Last Action
13. Logout
Enter your choice: 1

--- Add Book ---
ISBN: 9780134685992
Title: Java Testing Book
Author: Joshua Bloch
Categories: PROGRAMMING, DATABASE, NETWORKING, OPERATING_SYSTEM,
            MATHEMATICS, SCIENCE, HISTORY, FICTION
Category: PROGRAMMING
Publisher: Pearson
Publication Year: 2025
```

```
Total Copies: 5
Shelf Number: A - 15
Book added successfully!
---------------------------------------
Book ID          : BK011
ISBN             : 9780134685992
Title            : Java Testing Book
Author           : Joshua Bloch
Category         : PROGRAMMING
Publisher        : Pearson
Publication Year : 2025
Total Copies     : 5
Available Copies : 5
Shelf Number     : A - 15
---------------------------------------

--- Administrator Menu ---
 1. Add Book
 2. Update Book
 3. Delete Book
 4. Search Book
 5. View All Books
 6. Register Student
 7. Search Student
 8. View All Students
 9. Issue Book
10. Return Book
11. Generate Reports
12. Undo Last Action
13. Logout
Enter your choice: 2

--- Update Book ---
Book ID: BK011
Leave blank to keep current value.
ISBN [9780134685992]: 9780134685992
Title [Java Testing Book]: Java Testing Book
Author [Joshua Bloch]: Joshua Bloch
Categories: PROGRAMMING, DATABASE, NETWORKING, OPERATING_SYSTEM,
            MATHEMATICS, SCIENCE, HISTORY, FICTION
```

```
Category [PROGRAMMING]: PROGRAMMING
Publisher [Pearson]: Pearson
Publication Year [2025]: 2025
Total Copies [5]: 10
Shelf Number [A - 15]: A - 15
Book updated successfully!
----------------------------------------
Book ID          : BK011
ISBN             : 9780134685992
Title            : Java Testing Book
Author           : Joshua Bloch
Category         : PROGRAMMING
Publisher        : Pearson
Publication Year : 2025
Total Copies     : 10
Available Copies : 10
Shelf Number     : A - 15
----------------------------------------

--- Administrator Menu ---
 1. Add Book
 2. Update Book
 3. Delete Book
 4. Search Book
 5. View All Books
 6. Register Student
 7. Search Student
 8. View All Students
 9. Issue Book
10. Return Book
11. Generate Reports
12. Undo Last Action
13. Logout
Enter your choice: 4

--- Search Books ---
Enter search keyword (or press Enter for all): a
ID       Title                          Author              Category        Avail
--------------------------------------------------------------------------------
BK008    A Brief History of Time        Stephen Hawking     HISTORY         1
```

```
BK002    Clean Code                  Robert Martin        PROGRAMMING      2
BK004    Computer Networking         James Kurose         NETWORKING       1
BK007    Concepts of Physics         H.C. Verma           SCIENCE          2
BK003    Database System Concepts    Abraham Silberschatz DATABASE         2
BK006    Discrete Mathematics        Kenneth Rosen        MATHEMATICS      1
BK001    Effective Java              Joshua Bloch         PROGRAMMING      3
BK010    Java Testing Book           Joshua Bloch         PROGRAMMING      5
BK011    Java Testing Book           Joshua Bloch         PROGRAMMING      10
BK005    Operating System Concepts   Abraham Silberschatz OPERATING_SYSTEM 2
BK009    To Kill a Mockingbird       Harper Lee           FICTION          2

--- Administrator Menu ---
 1. Add Book
 2. Update Book
 3. Delete Book
 4. Search Book
 5. View All Books
 6. Register Student
 7. Search Student
 8. View All Students
 9. Issue Book
10. Return Book
11. Generate Reports
12. Undo Last Action
13. Logout
Enter your choice: 5

--- All Books ---
ID       Title                       Author               Category         Avail
-------------------------------------------------------------------------------
BK008    A Brief History of Time     Stephen Hawking      HISTORY          1
BK002    Clean Code                  Robert Martin        PROGRAMMING      2
BK004    Computer Networking         James Kurose         NETWORKING       1
BK007    Concepts of Physics         H.C. Verma           SCIENCE          2
BK003    Database System Concepts    Abraham Silberschatz DATABASE         2
BK006    Discrete Mathematics        Kenneth Rosen        MATHEMATICS      1
BK001    Effective Java              Joshua Bloch         PROGRAMMING      3
BK010    Java Testing Book           Joshua Bloch         PROGRAMMING      5
BK011    Java Testing Book           Joshua Bloch         PROGRAMMING      10
```

```
--- Administrator Menu ---
 1. Add Book
 2. Update Book
 3. Delete Book
 4. Search Book
 5. View All Books
 6. Register Student
 7. Search Student
 8. View All Students
 9. Issue Book
10. Return Book
11. Generate Reports
12. Undo Last Action
13. Logout
Enter your choice: 6

--- Register Student ---
Name: Tejesh
Department: CSE
Phone (10 digits): 720563859
Email: tejeshhhk@gmail.com
Error: Phone number must be exactly 10 digits.

--- Administrator Menu ---
 1. Add Book
 2. Update Book
 3. Delete Book
 4. Search Book
 5. View All Books
 6. Register Student
 7. Search Student
 8. View All Students
 9. Issue Book
10. Return Book
11. Generate Reports
12. Undo Last Action
13. Logout
Enter your choice: 6
```

```
--- Register Student ---
Name: tejesh
Department: CSE
Phone (10 digits): 7207563859
Email: tejeshhhk@gmail.com
Student registered successfully!
--------------------------------------
Student ID   : ST004
Name         : tejesh
Department   : CSE
Phone        : 7207563859
Email        : tejeshhhk@gmail.com
Borrowed IDs : None
--------------------------------------

--- Administrator Menu ---
 1. Add Book
 2. Update Book
 3. Delete Book
 4. Search Book
 5. View All Books
 6. Register Student
 7. Search Student
 8. View All Students
 9. Issue Book
10. Return Book
11. Generate Reports
12. Undo Last Action
13. Logout
Enter your choice: 7

--- Search Students ---
Enter search keyword (or press Enter for all):
ID       Name                     Department      Borrowed
-----------------------------------------------------------------
ST002    Priya Sharma             Information ... 0
ST003    Rahul Verma              Electronics     0
ST001    Tejesh Kumar             Computer Sci... 0
ST004    tejesh                   CSE             0
```

```
--- Administrator Menu ---
 1. Add Book
 2. Update Book
 3. Delete Book
 4. Search Book
 5. View All Books
 6. Register Student
 7. Search Student
 8. View All Students
 9. Issue Book
10. Return Book
11. Generate Reports
12. Undo Last Action
13. Logout
Enter your choice: 8

--- All Students ---
ID       Name                     Department      Borrowed
-----------------------------------------------------------
ST002    Priya Sharma             Information ... 0
ST003    Rahul Verma              Electronics     0
ST001    Tejesh Kumar             Computer Sci... 0
ST004    tejesh                   CSE             0

--- Administrator Menu ---
 1. Add Book
 2. Update Book
 3. Delete Book
 4. Search Book
 5. View All Books
 6. Register Student
 7. Search Student
 8. View All Students
 9. Issue Book
10. Return Book
11. Generate Reports
12. Undo Last Action
13. Logout
Enter your choice: 9
```

```
--- Issue Book ---
Student ID: ST004
Book ID: BK004
Book issued successfully!
----------------------------------------
Issue ID     : IS002
Student ID   : ST004
Book ID      : BK004
Issue Date   : 2026-07-15
Due Date     : 2026-07-29
Return Date  : Not Returned
Fine Amount  : Rs. 0.00
----------------------------------------

--- Administrator Menu ---
 1. Add Book
 2. Update Book
 3. Delete Book
 4. Search Book
 5. View All Books
 6. Register Student
 7. Search Student
 8. View All Students
 9. Issue Book
10. Return Book
11. Generate Reports
12. Undo Last Action
13. Logout
Enter your choice: 10

--- Return Book ---
Student ID: ST004
Book ID:  BK004
Book returned successfully!
----------------------------------------
Issue ID     : IS002
Student ID   : ST004
Book ID      : BK004
Issue Date   : 2026-07-15
```

```
Book ID      : BK004
Issue Date   : 2026-07-15
Due Date     : 2026-07-29
Return Date  : 2026-07-15
Fine Amount  : Rs. 0.00
----------------------------------------

--- Administrator Menu ---
 1. Add Book
 2. Update Book
 3. Delete Book
 4. Search Book
 5. View All Books
 6. Register Student
 7. Search Student
 8. View All Students
 9. Issue Book
10. Return Book
11. Generate Reports
12. Undo Last Action
13. Logout
Enter your choice: 11

========================================
       LIBRARY MANAGEMENT REPORT
========================================
Report Date : 2026-07-15
----------------------------------------
BOOK STATISTICS
  Total Book Titles     : 11
  Total Copies          : 31
  Available Titles      : 11
  Available Copies      : 31
  Issued Copies         : 0
  Overdue Issues        : 0

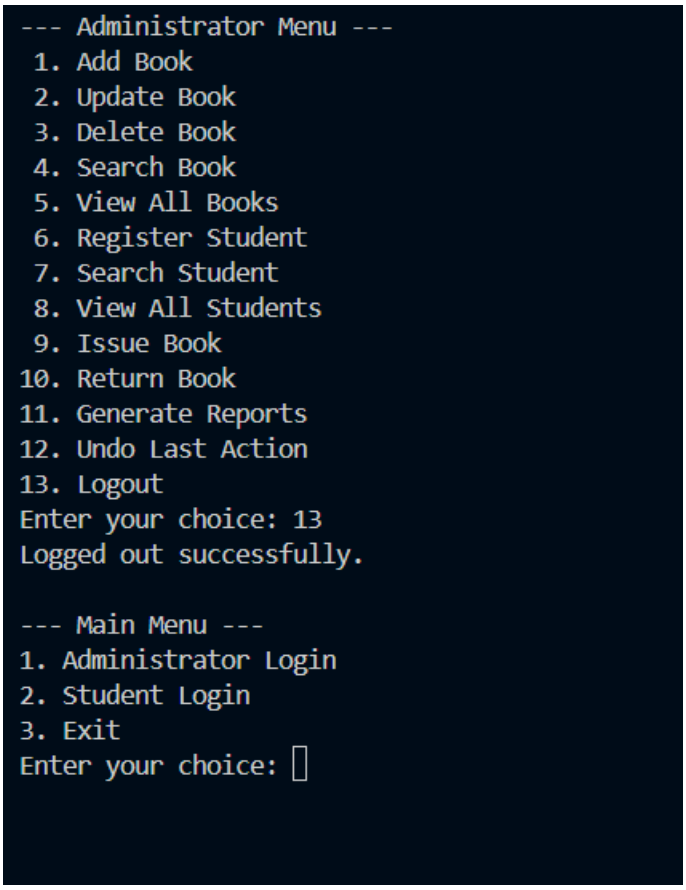
STUDENT STATISTICS
  Total Students        : 4
  Active Students       : 0
```

```
FINANCIAL
  Total Fine Collected  : Rs. 0.00
=======================================


--- Administrator Menu ---
 1. Add Book
 2. Update Book
 3. Delete Book
 4. Search Book
 5. View All Books
 6. Register Student
 7. Search Student
 8. View All Students
 9. Issue Book
10. Return Book
11. Generate Reports
12. Undo Last Action
13. Logout
Enter your choice: 12

--- Undo Last Action ---
Undo successful: Returned book BK004 from student ST004

--- Administrator Menu ---
 1. Add Book
 2. Update Book
 3. Delete Book
 4. Search Book
 5. View All Books
 6. Register Student
 7. Search Student
 8. View All Students
 9. Issue Book
10. Return Book
11. Generate Reports
12. Undo Last Action
13. Logout
Enter your choice: 13
    >>
```


--- Administrator Menu ---
1. Add Book
2. Update Book
3. Delete Book
4. Search Book
5. View All Books
6. Register Student
7. Search Student
8. View All Students
9. Issue Book
10. Return Book
11. Generate Reports
12. Undo Last Action
13. Logout
Enter your choice: 13
Logged out successfully.
--- Main Menu ---
1. Administrator Login
2. Student Login
3. Exit
Enter your choice: